Session 7 · Deck 7.A1

# The Learning Problem: a Handwritten Digit, and the Neuron

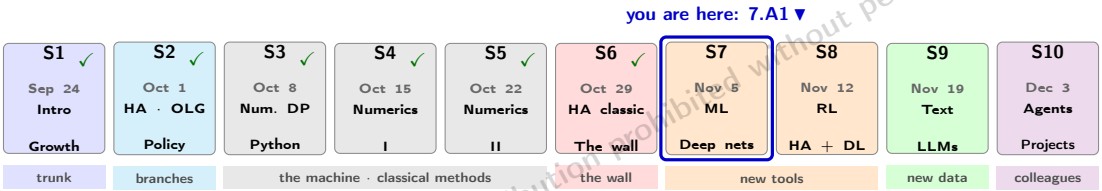
Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng Redistribution prohibited without permission

*Build it on paper → solve it on the machine → push it to the frontier with AI*

# Where We Are




*Build it on paper → solve it on the machine → push it to the frontier with AI*

**Block A — learning a function: 7.A1** the learning problem and the neuron · 7.A2 activations and approximation · 7.A3 backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · 7.B2 the growth model · 7.B3 architecture and shape · 7.B4 capacity and order.

© Zhiang Lou Redistribution prohibited without permission

# Roadmap: Three Parts

**Sessions 3 to 6 solved models on grids with a basis chosen in advance. This deck starts one step earlier: with a problem no formula solves.**

| Part | Topic | The question it answers |
|---|---|---|
| 1 | Session 7, and where it starts | What will you be able to do, and what do you already have? |
| 2 | A handwritten digit | What is a learning problem, and why can it be solved at all? |
| 3 | The neuron | What is the smallest unit that learns, and what can it not do? |

**Idea**

A learning problem is data, a family of functions $f_\theta$, a loss, and a rule for judging the result. A neuron is the smallest member of the family: a logit.

© Zhaoyi Feng
Redistribution prohibited without permission

# 1 · What You Will Be Able to Do After Session 7

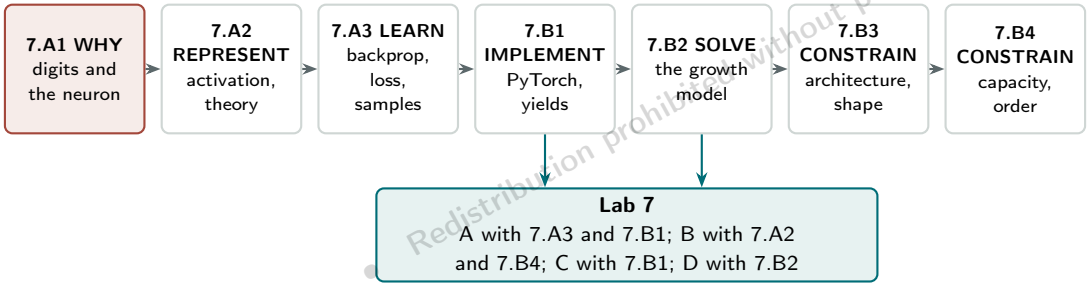
**Six skills, in the order the seven decks build them.**

- **Read a network as a function**: write $f_\theta(x)$, count its parameters, and say what its architecture assumes about the input (7.A1, 7.A2, 7.B3, 7.B4).
- **Check one training step by hand against autograd**: forward, loss, backward, update (7.A3, 7.B1).
- **Write the PyTorch training loop** and check it on a problem whose answer you already know (7.B1).
- **Run an honest forecast comparison**: splits by date, one information set, a random-walk baseline (7.B1).
- **Solve a growth model with a network**, and show with held-out tests that the solution is right (7.B2).
- **Throughout: a small training loss is a claim about the training sample, and nothing more.**

© Zhigang Feng Redistribution prohibited without permission

# 1 · Session 7 in One Map

**Seven decks: why, represent, learn; then implement, solve, constrain twice.**




Redistribution prohibited without permission

© Zhigang Feng

- The running example for 7.A1 to 7.A3 is one picture: a handwritten digit.
- The economic applications are in 7.B1 (a forecast) and 7.B2 (a model solved).
- **One habit in all seven decks: test a method first where the answer is known.**

# 1 · What Sessions 1–6 Gave Us, and What Is New Here

**The machine you already have, and the four ideas this session adds to it.**

**What you already have**

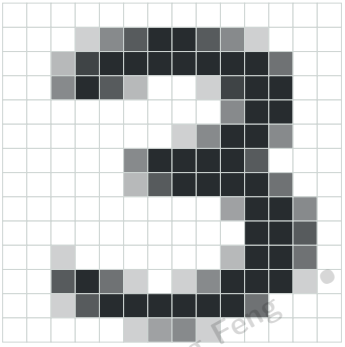
- **Projection** (5.B1): a few numbers $\theta$ stand in for a function. The basis is fixed; only $\theta$ is chosen.
- **A loss for a model**: the Euler residual, which bounds the policy error (3.A1) and grades a solution (3.A3).
- **Derivatives without algebra**: autograd, and a first PyTorch training loop (3.B3).
- **An optimiser for a sum over data**: SGD, a mini-batch gradient with a shrinking step (4.A2).
- **The reason**: when the distribution is a state, grids run out (1.B3, 6.B1) — 1.A3's "solving models we could not solve".

**What is new today**

- **Supervised learning**: learn a function from inputs paired with labels (reinforcement learning, 1.B3, is Session 8).
- **Empirical risk minimisation**: choose $\theta$ to minimise $\frac{1}{n}\sum_i \ell\big(f_\theta(x_i), y_i\big)$. OLS is the linear, squared-loss case.
- Judge a model on data it never saw. 7.A3 makes this a procedure.
- The network as a basis that is itself *learned* — 5.B1's promise, kept in 7.A2 and 7.B2.

© Zhiming Feng · Redistribution prohibited without permission

**One picture carries the ideas of 7.A1 to 7.A3.**

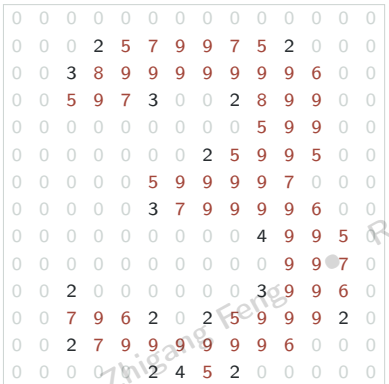


- You read this as a "3" at once, and you cannot say how.
- The benchmark is **MNIST**: 70,000 digits written by US Census Bureau staff and high-school students, each a $28 \times 28$ grey-scale image with its label.
- Drawn here at $14 \times 14$, so that every pixel can be seen.
- **Nobody writes that function down. It is learned from labelled examples.**

Redistribution prohibited without permission
© Zhigang Feng

Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition", *Proc. IEEE* 86(11), 1998.

**To the machine the picture is a table of numbers, and the table is a vector.**

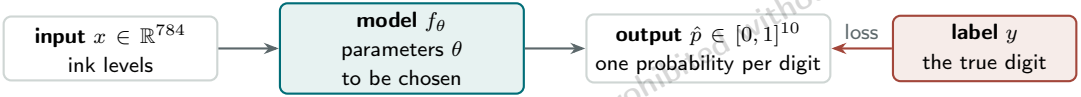


- To a computer the image is a table of ink levels: 0 is blank paper, 9 is full ink (0 to 255 in the original files).
- Read row after row, the table is a vector: 196 numbers here, $28 \times 28 = 784$ in MNIST. That vector is the entire input.
- Flattening forgets which pixels are neighbours. 7.B3 puts that back.
- **An image is a point in a space with one dimension per pixel: 196 here, 784 in MNIST.**

© Zhuang Feng. Redistribution prohibited without permission

# The Learning Problem: from 784 Numbers to Ten Classes

**Data, a model, a loss, and learning: the econometrician's problem with a richer $f_\theta$.**




© Zhigang Feng · Redistribution prohibited without permission

- **Data:** 60,000 labelled pairs $(x_i, y_i)$ for training, 10,000 held out for testing.
- **Model:** a family of functions $f_\theta$ indexed by a parameter vector $\theta$.
- **Loss:** small when $\hat{p}$ puts high probability on the true digit.
- **Learning:** choose $\theta$ to make the average training loss small, then measure on the held-out images.
- **The econometrician's problem with a richer $f_\theta$: a specification, a criterion, an estimator.**

# Try Writing the Rule for a "3"

**When the rule cannot be written down, give examples and let it be estimated.**



four people, one digit

- A first rule: "two open curves, stacked". Now define *curve*, *open* and *stacked* in ink levels.
- Slant, stroke width, size and position change all 784 numbers and none changes the digit.
- **Polanyi's paradox**: we know more than we can tell.
- **When the rule cannot be written, give examples and let the rule be estimated.**

© Zhigang Feng
Redistribution prohibited without permission

M. Polanyi, *The Tacit Dimension*, 1966; D. Autor, "Polanyi's paradox and the shape of employment growth", NBER WP 20485, 2014.

# Why It Is Possible at All: Structure

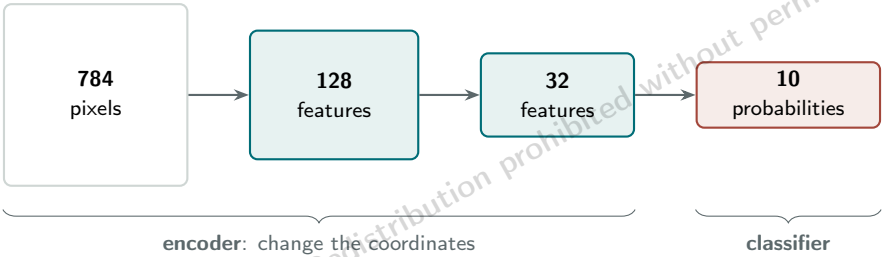
**Real data occupy a small, structured part of the space they are recorded in.**

- There are $256^{784}$ possible $28 \times 28$ images. Almost all of them look like television static.
- Images of digits are a vanishing fraction of that space, and they are not scattered at random: strokes are continuous, connected and few.
- The **manifold hypothesis**: real high-dimensional data lie near a surface of much lower dimension inside the space they are recorded in.
- Macroeconomists use the same idea. Krusell and Smith (1998) replace an infinite-dimensional wealth distribution by its mean, because in equilibrium the distribution moves in very few directions (6.B1).
- **Learning in high dimension works when the data have low-dimensional structure, and fails when they do not.**

© Zhiyang Feng · Redistribution prohibited without permission

P. Krusell and A. Smith, "Income and wealth heterogeneity in the macroeconomy", *JPE* 106(5), 1998.

**A deep network is a logit whose explanatory variables are learned.**

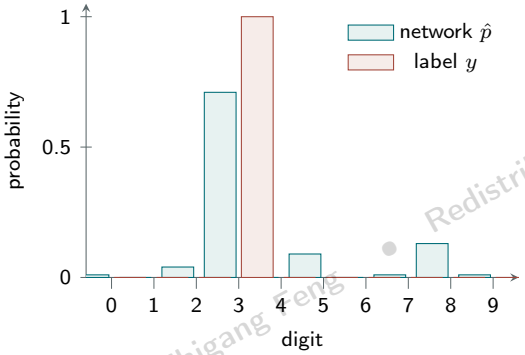



© Zhirang Feng · Redistribution prohibited without permission

- Each layer re-describes the input in fewer, more useful numbers. The last layer is an ordinary multinomial logit on those numbers.
- The encoder and the classifier are estimated *jointly*, by the same criterion.
- In econometric terms: the regressors are estimated, not chosen by the researcher.
- **A deep network is a logit whose explanatory variables are learned.**

# Labels, One-Hot Targets, and a Probability for Each Digit

**Classification is maximum likelihood for a discrete-choice model.**




An illustrative $\hat{p}$, not a trained network's output.

- The label "3" is a **one-hot** vector: a 1 in position 3, zeros elsewhere.
- The **softmax** turns ten scores $z_j$ into probabilities, $\hat{p}_j = e^{z_j} / \sum_k e^{z_k}$: McFadden's multinomial logit, with scores as utilities.
- The loss is the negative log-likelihood of the true digit: $-\log 0.71 = 0.34$ here.
- **Classification is maximum likelihood for a discrete-choice model.**

Redistribution prohibited without permission
© Zhigang Feng

**An architecture is an assumption about the input, written into the function.**

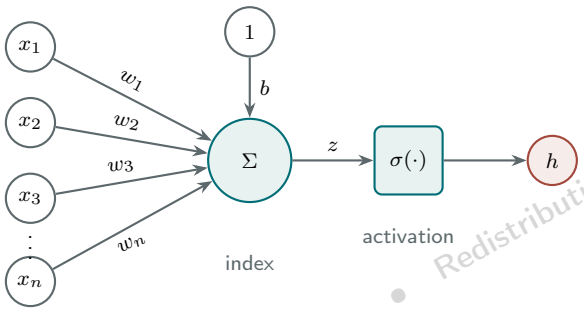
| Input | Structure it carries | Architecture | Economic example |
|---|---|---|---|
| image | neighbours matter | convolutional | night lights |
| sequence | order matters | recurrent, transformer | policy statements |
| set, network | order is irrelevant | graph network | households |
| state vector | none assumed | dense | $(k, z)$ in a growth model |

© Zhigang Feng Redistribution prohibited without permission

- 7.A1 to 7.B2 use the dense network, which assumes nothing. 7.B3 and 7.B4 return to the other rows.
- **An architecture is an assumption about the input, written into the function.**

# A Neuron Is a Regression with a Nonlinear Output

**A linear index, then a fixed nonlinear function: a generalised linear model.**

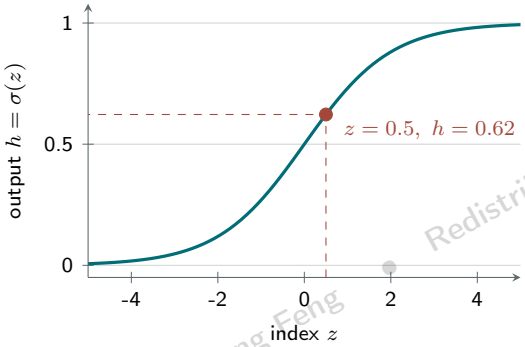



- Step 1, a linear index: $z = w^\top x + b$. The **weights** $w$ are slopes; the **bias** $b$ is the intercept.
- Step 2, a fixed nonlinear function, the **activation**: $h = \sigma(z)$.
- With $\sigma(z) = 1/(1 + e^{-z})$ this is exactly a logit model.
- The parameters are $\theta = (w, b)$: $n + 1$ numbers.
- **One neuron is a generalised linear model. Nothing more.**

© Zhigang Feng
Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.1.

**Three inputs, three weights, one bias: the neuron is "62% on".**

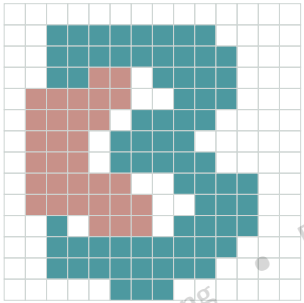



© Zhigang Feng

Redistribution prohibited without permission

- Inputs $x = (1.0,\ 0.5,\ 0.0)$; weights $w = (0.8,\ -0.4,\ 0.3)$; bias $b = -0.1$.
- Index: $z = 0.8(1.0) - 0.4(0.5) + 0.3(0.0) - 0.1 = 0.5$.
- Output: $h = \sigma(0.5) = 1/(1 + e^{-0.5}) = 0.62$.
- Read it as a probability: the neuron is "62% on".
- The third input is zero, so $w_3$ played no part. **A weight matters only where its input is active.**

**One neuron asks one question of the input: how much do you look like my template?**

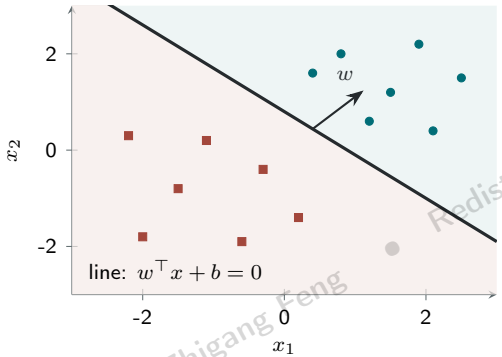


**teal**: positive weight **red**: negative weight

- A neuron reading the image has one weight per pixel, so its weights can be drawn as an image.
- $w^\top x$ is large when ink falls on positive weights and stays off negative ones.
- This one rewards ink where a "3" has strokes and penalises ink on the left, where an "8" closes its loops.
- **One neuron asks one question of the input: how much do you look like my template?**

© Zhigang Feng Redistribution prohibited without permission

**The neuron switches on in a half-space; its edge is a straight line.**




- $\sigma(z) > 0.5$ exactly when $z = w^\top x + b > 0$.
- The neuron switches on in a half-space whose edge is the **hyperplane** $w^\top x + b = 0$: a line in two dimensions.
- $w$ is perpendicular to the line; $b$ slides it. The activation sets how sharply the output changes across the line, and does not bend it.
- **One neuron can separate only what a single straight cut can separate.**

© Zhigang Feng
Redistribution prohibited without permission

# An Economist Has Met This Before: Logit

**Same unit, same criterion, different names. What is new is stacking.**

- **Binary choice.** $\Pr(y = 1 \mid x) = \sigma(w^\top x + b)$ is the logit model; $w^\top x + b$ is the difference in systematic utility.
- **Estimation.** Maximum likelihood on the logit is minimising **cross-entropy**, the loss used to train classifiers. Same criterion, different name.
- **Vocabulary.** weights = coefficients; bias = intercept; activation = link function; training = estimation; inference = prediction.
- **What is new** is not the unit. It is stacking many units so that the regressors of the last one are themselves estimated.
- **Everything you know about specification, identification and out-of-sample testing still applies.**

© Zhigang Feng · Redistribution prohibited without permission

D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

# Takeaways

- **Part 1.** Session 7 is supervised learning, for two jobs: predict (7.B1) and compute (7.B2). A small training loss is a claim about the training sample, and nothing more.
- **Part 2.** A learning problem is data, a family $f_\theta$, a loss, and a rule for judging the result. An image is a vector, and learning works because real data occupy a small, structured part of the space.
- **Part 3.** A neuron is a logit: a linear index through a fixed nonlinearity, $h = \sigma(w^\top x + b)$. Alone it draws one line, and one line is not enough for the digits or for XOR.

© Zhigang Feng — Reproduction prohibited without permission

# Bridge: The Piece We Have Not Looked At

The whole of a network's flexibility sits in that $\sigma$. We have used it without looking at it.

Take it away and a stack of layers collapses into one regression. Keep it and a network with one hidden layer can reach any continuous function on a bounded domain — which is a promise about existence, not about finding it.

**Next: 7.A2**

What $\sigma$ is, why a layer without it buys nothing, and what a network with it can reach: XOR exactly, $\sin x$ out of hinges, and the theorems that say how far.

© Zhigang Feng · Redistribution prohibited without permission